

CREATIVE ADHYAYAN

# LMS Function Tree & Technical Architecture

A detailed role, feature, workflow, service, and database function map

Repository snapshot: 26 September 2026

## Scope

This document explains what each major LMS function does, how roles interact with it, which services and database functions support it, and where the implementation lives. Small visual-only helpers are summarized under their parent screen; reusable and security-relevant functions are cataloged individually.

# 1. System map

The application is a React/Vite client backed by Supabase Auth, PostgreSQL, Row Level Security, Storage, Realtime, and Edge Functions.

| Creative Adhyayan LMS      |  | Purpose / downstream behavior                                                                                  |
|----------------------------|--|----------------------------------------------------------------------------------------------------------------|
| -- Public experience       |  | Home, short courses, course details, authentication, contact, and legal pages.                                 |
| -- Student workspace       |  | Purchased courses, learning, assignments, attendance, payments, certificates, profile, and community.          |
| -- Teacher workspace       |  | Course creation, curriculum, lessons, assignments, batches, attendance, offline admissions, and announcements. |
| -- Administrator workspace |  | Students, course approval, payments/refunds, exports, analytics, administrators, and announcements.            |
| -- Application services    |  | Translate UI actions into validated database, storage, payment, and notification operations.                   |
| -- Supabase database       |  | Canonical LMS tables, RPC functions, triggers, constraints, and role-aware RLS policies.                       |
| -- Edge Functions          |  | Razorpay order creation, capture verification, webhooks, EMI scheduling, refunds, and receipts.                |

## Core request path

| UI to protected data                    | Result                                                                                            |
|-----------------------------------------|---------------------------------------------------------------------------------------------------|
| 1. A user performs an action in a route | React validates immediate input and shows progress/errors.                                        |
| 2. A service function runs              | The service normalizes identifiers and calls Supabase tables, RPCs, Storage, or an Edge Function. |
| 3. Database authorization executes      | RLS, constraints, and security-definer RPC validation determine whether the action is allowed.    |
| 4. Canonical records are written        | Related dashboards read the same tables so teacher, student, and admin views stay aligned.        |
| 5. Realtime/refetch updates the UI      | Listeners and explicit refreshes display the latest state.                                        |

## 2. Public and authentication functions

| Public visitor                   | Purpose / downstream behavior                                                                         |
|----------------------------------|-------------------------------------------------------------------------------------------------------|
| -- Browse home and short courses | Reads published courses and promotional content without exposing private course data.                 |
| -- Open course details           | Loads published outline, pricing, teacher, preview lessons, and current enrollment state.             |
| -- Register                      | Creates a student identity/profile and routes authentication verification.                            |
| -- Login                         | Authenticates email/password, phone/password, or configured provider and restores the intended route. |
| -- Forgot/reset password         | Requests a recovery email and updates credentials after a valid recovery callback.                    |
| -- Contact                       | Validates and submits a support/contact request.                                                      |
| -- Legal information             | Shows Terms, Privacy Policy, and Refund/Cancellation Policy.                                          |

### Route protection

| AccessGate        | Purpose / downstream behavior                                                        |
|-------------------|--------------------------------------------------------------------------------------|
| -- RequireAuth    | Allows any authenticated, active LMS account.                                        |
| -- RequireStudent | Requires the student role.                                                           |
| -- RequireTeacher | Requires the teacher role.                                                           |
| -- RequireAdmin   | Requires the administrator role.                                                     |
| -- Redirect logic | Sends unauthenticated users to login and role mismatches to their correct dashboard. |

### 3. Student function tree

| Student dashboard       | Purpose / downstream behavior                                                                         |
|-------------------------|-------------------------------------------------------------------------------------------------------|
| -- My courses           | Lists active enrollments and provides entry into course learning.                                     |
| -- Learn course         | Loads accessible modules/lessons, resumes progress, protects locked modules, and updates last access. |
| -- Video watch tracking | Records bounded watch-time chunks and video completion before allowing lesson completion.             |
| -- Lesson completion    | Server validates access/watch requirements and updates completed lessons and progress.                |
| -- Automatic attendance | Creates one present record per course/day after an active student opens a lesson.                     |
| -- Course community     | Reads and posts course discussions available to enrolled learners.                                    |
| -- Assignments          | Views visible assignments, submits work, and reads grade/feedback.                                    |
| -- Attendance           | Shows today's status, history, present/late/absent totals, and percentage.                            |
| -- Payments             | Displays verified payment history, payment mode, amount, and status.                                  |
| -- Certificates         | Lists earned certificates and generates/downloads certificate PDFs.                                   |
| -- Profile              | Reads and updates allowed learner profile fields and avatar.                                          |

| Automatic daily attendance                        | Result                                                                                  |
|---------------------------------------------------|-----------------------------------------------------------------------------------------|
| 1. Student opens an accessible lesson             | LearnCourse confirms an active enrollment and selected lesson.                          |
| 2. Client calls<br>markCourseEngagementAttendance | The request contains only the course ID; identity comes from the authenticated session. |
| 3. RPC validates enrollment and course            | Inactive/non-enrolled users are rejected; the published course supplies the teacher ID. |
| 4. Daily closed session is upserted               | ID: engagement_{course}_{date}; metadata identifies automatic course engagement.        |
| 5. Present record is upserted                     | Unique session/student constraint prevents duplicates across lessons and refreshes.     |
| 6. Dashboards read canonical attendance           | The teacher report and student summary show the same record.                            |

## 4. Course access and payment tree

| Enrollment and access         | Purpose / downstream behavior                                                                                          |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------|
| -- Free enrollment            | lms_enroll_free creates an active enrollment after server validation.                                                  |
| -- One-time payment           | Edge Function calculates the authoritative price, creates Razorpay order, verifies capture, and activates paid access. |
| -- Full-payment module access | All course modules unlock immediately; migration 202609260004 also repairs earlier paid enrollments.                   |
| -- EMI enrollment             | Registration fee creates/activates a plan and unlocks modules according to paid stages and schedule rules.             |
| -- Self-paced EMI             | Modules map to registration/installment stages without requiring a batch calendar.                                     |
| -- Scheduled EMI              | Paid stage plus scheduled date determines module availability.                                                         |
| -- Overdue state              | Reminder/sync logic marks the plan overdue and enrollment payment_due until payment continues.                         |
| -- Refund                     | Admin requests an eligible Razorpay refund; processed refunds revoke one-time course access.                           |

| Verified one-time purchase                  | Result                                                                                 |
|---------------------------------------------|----------------------------------------------------------------------------------------|
| 1. CourseDetails requests create-order      | Server reads course price and applies the configured direct-payment discount.          |
| 2. Razorpay Checkout completes              | Client receives order/payment/signature references, never the Razorpay secret.         |
| 3. verify-payment validates signature       | Server fetches/captures payment and checks order, amount, currency, and status.        |
| 4. lms_confirm_payment runs transactionally | Payment becomes paid, enrollment becomes active/paid, and module access is recomputed. |
| 5. Receipt is attempted                     | Receipt failure is logged but cannot undo a verified enrollment.                       |
| 6. Webhook provides recovery                | A payment.captured event completes activation if the browser closes early.             |

## 5. Teacher function tree

| Teacher workspace      | Purpose / downstream behavior                                                                           |
|------------------------|---------------------------------------------------------------------------------------------------------|
| -- Dashboard           | Summarizes owned courses, learners, batches, assignments, and recent activity.                          |
| -- Course lifecycle    | Creates drafts, edits allowed fields, submits for review, and sees approval/rejection state.            |
| -- Curriculum builder  | Creates, reorders, updates, and deletes modules and lessons.                                            |
| -- Lesson editor       | Configures video/resource content, preview visibility, publication, duration, and uploads.              |
| -- Publishing boundary | Teacher submits; admin performs final publication. Database policy prevents teacher self-publication.   |
| -- Assignments         | Creates assignments, targets a course/batch, publishes, reviews submissions, grades, and adds feedback. |
| -- Batches             | Creates teaching groups, assigns active enrolled students, stores schedule, and maintains roster.       |
| -- Batch attendance    | For a selected date, marks every roster student present, late, or absent and saves canonical records.   |
| -- Attendance monitor  | Aggregates owned-course attendance and supports student/course/batch/status search.                     |
| -- Offline admissions  | Looks up or creates eligible students and records verified offline course admission.                    |
| -- Announcements       | Publishes teacher-owned course announcements to permitted audiences.                                    |

| Teacher batch attendance               | Result                                                                |
|----------------------------------------|-----------------------------------------------------------------------|
| 1. Teacher opens a batch and date      | Roster is restricted to active students enrolled in the batch course. |
| 2. Teacher selects present/late/absent | Client validates that every status is allowed.                        |
| 3. Batch attendance document is saved  | One batch/date record stores the status map.                          |
| 4. Canonical closed session is saved   | The teacher and student attendance screens use this shared session.   |
| 5. Individual records are upserted     | One row per student supports reporting and exports.                   |

## 6. Administrator function tree

| Administrator workspace | Purpose / downstream behavior                                                                          |
|-------------------------|--------------------------------------------------------------------------------------------------------|
| -- Dashboard            | Displays system-level enrollment, revenue, course, teacher, and learner indicators.                    |
| -- Students             | Searches learners, inspects enrollments/payments, and changes permitted account status.                |
| -- Courses              | Reviews submitted courses, publishes/rejects/archives, and controls featuring/order.                   |
| -- Payments             | Reads normalized payment history, inspects details, and initiates supported refunds.                   |
| -- Exports              | Produces filtered operational datasets for students, courses, payments, attendance, and notifications. |
| -- Analytics            | Aggregates active enrollments, captured revenue, trends, rankings, and category distribution.          |
| -- Admins               | Creates/manages administrator access within protected role-change rules.                               |
| -- Announcements        | Creates global or targeted institutional announcements.                                                |

### Administrative safety boundaries

| Defense in depth          | Purpose / downstream behavior                                                       |
|---------------------------|-------------------------------------------------------------------------------------|
| -- Role gate              | Admin routes require an authenticated admin profile.                                |
| -- RLS policy             | Database checks the active profile role for every protected table operation.        |
| -- Record guard trigger   | Immutable ownership fields and privileged status changes are protected.             |
| -- Service-role boundary  | Payment confirmation, webhook processing, and sensitive mutations stay server-side. |
| -- Audit-friendly records | Payments, enrollments, attendance, and exports preserve source IDs and timestamps.  |

## 7. Communication, learning, and records

| Supporting feature tree | Purpose / downstream behavior                                                                   |
|-------------------------|-------------------------------------------------------------------------------------------------|
| -- Announcements        | Role-aware publishing and audience filtering for global/course/enrolled learners.               |
| -- Notifications        | Recipient-specific messages with read state and action URLs.                                    |
| -- Course community     | Enrollment-gated discussions and replies.                                                       |
| -- Assignments          | Visibility rules combine ownership, publication, enrollment, and optional batch targeting.      |
| -- Progress             | Completed lesson IDs, percentage, last lesson, and last access are stored on enrollment.        |
| -- Certificates         | Issued certificate records bind student, course, enrollment, name, and issue date.              |
| -- Media storage        | Uploads use controlled paths; signed/refreshed URLs are normalized before UI consumption.       |
| -- Realtime             | Database changes trigger refetch while a timed fallback covers deployments without replication. |

### Canonical data ownership

| Supabase tables                                             | Purpose / downstream behavior                                                        |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------|
| -- lms_profiles                                             | Identity-linked name, email, role, status, avatar, phone, and role-change timestamp. |
| -- lms_courses / modules / lessons                          | Course catalog and ordered learning content.                                         |
| -- lms_enrollments / payments                               | Access, progress, payment status, and provider references.                           |
| -- lms_installment_plans / emi_installments / module_access | EMI schedule, paid stages, and module-level unlocking.                               |
| -- lms_batches / batch_attendance                           | Teacher groups, roster, schedule, and batch/date status maps.                        |
| -- lms_attendance_sessions / attendance_records             | Canonical daily/session attendance used by all dashboards.                           |
| -- lms_assignments / assignment_submissions                 | Assessment lifecycle, work, grades, and feedback.                                    |
| -- announcements / notifications / discussions              | Communication and community activity.                                                |

## 8. Security and integrity tree

| Security controls        | Purpose / downstream behavior                                                                                 |
|--------------------------|---------------------------------------------------------------------------------------------------------------|
| -- Authentication        | Supabase session establishes auth.uid(); client-supplied identity is not trusted for sensitive RPCs.          |
| -- Active profile role   | lms_role returns a role only for active accounts.                                                             |
| -- Ownership predicates  | lms_admin, lms_owns_course, lms_enrolled, and lms_course_member centralize authorization.                     |
| -- Row Level Security    | Every LMS table defines role- and ownership-aware select/write policies.                                      |
| -- Guard trigger         | Rejects illegal role changes, ownership changes, self-publishing, self-grading, and direct enrollment writes. |
| -- Security-definer RPCs | Perform narrowly scoped trusted mutations after explicit validation.                                          |
| -- Payment verification  | Signature, captured status, amount, currency, and order ownership are checked server-side.                    |
| -- Idempotency           | Unique IDs/constraints prevent duplicate enrollment, attendance, installments, and webhook effects.           |
| -- Storage authorization | Paths and membership checks control media access.                                                             |

### Important invariants

| Rules that must remain true | Purpose / downstream behavior                                                              |
|-----------------------------|--------------------------------------------------------------------------------------------|
| -- Enrollment               | Paid access is granted only after server verification; free access uses the dedicated RPC. |
| -- Full payment             | All modules unlock immediately and cannot inherit stale EMI locks.                         |
| -- EMI                      | Only paid stages unlock their assigned modules; overdue plans can suspend access.          |
| -- Attendance               | A learner can create only their own present record through approved flows.                 |
| -- Assignments              | Students cannot grade; teachers can operate only within owned courses.                     |
| -- Publishing               | Teachers prepare and submit; administrators approve publication.                           |
| -- Refunds                  | Access revocation follows confirmed refund processing, not merely a refund request.        |

## 9. Function catalog

Reusable functions and primary UI components are listed with signatures, responsibilities, and source locations. SQL entries show the latest migration definition when a function was replaced over time.

### Application services - AdminCourseService

| Function                                     | Responsibility                                                                                                                                                                                            | Source                                  |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| <code>requireAuthenticatedAdmin()</code>     | ADMIN AUTH CHECK                                                                                                                                                                                          | src/services/AdminCourseService.js:41   |
| <code>courseFromDoc(courseDoc)</code>        | HELPERS                                                                                                                                                                                                   | src/services/AdminCourseService.js:56   |
| <code>sortCourses(a, b)</code>               | Implements sort courses for the AdminCourse service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                 | src/services/AdminCourseService.js:64   |
| <code>notifyCourseTeacher({})</code>         | Sends a notification to the teacher who owns a course. Notification failure must NOT break the main admin action. Example: Admin approves course -> Course becomes published -> Teacher gets notification | src/services/AdminCourseService.js:102  |
| <code>getAllCourses()</code>                 | Gets every course in the LMS. Admin can see: draft pending published rejected archived                                                                                                                    | src/services/AdminCourseService.js:167  |
| <code>getPendingCourses()</code>             | Gets courses waiting for admin approval.                                                                                                                                                                  | src/services/AdminCourseService.js:187  |
| <code>subscribeToAllCourses()</code>         | Real-time listener for the Admin Course table.                                                                                                                                                            | src/services/AdminCourseService.js:212  |
| <code>approveCourse()</code>                 | Admin approves a pending course. pending -> published Teacher receives: "Course approved"                                                                                                                 | src/services/AdminCourseService.js:269  |
| <code>adminPublishCourse()</code>            | Admin publishes a draft course directly, skipping the pending-review step. draft -> published Teacher receives: "Course published"                                                                        | src/services/AdminCourseService.js:361  |
| <code>rejectCourse()</code>                  | Admin rejects a course. pending -> rejected Teacher receives: "Course rejected"                                                                                                                           | src/services/AdminCourseService.js:452  |
| <code>adminUnpublishCourse()</code>          | Emergency admin action. published -> archived Teacher receives notification.                                                                                                                              | src/services/AdminCourseService.js:552  |
| <code>sendArchivedCourseForApproval()</code> | archived -> pending Teacher receives notification.                                                                                                                                                        | src/services/AdminCourseService.js:627  |
| <code>setFeatured()</code>                   | Admin controls whether a course appears in the featured/homepage area.                                                                                                                                    | src/services/AdminCourseService.js:701  |
| <code>reorderFeaturedCourses()</code>        | Admin controls homepage order.                                                                                                                                                                            | src/services/AdminCourseService.js:753  |
| <code>setCourseOrder()</code>                | SET COURSE ORDER                                                                                                                                                                                          | src/services/AdminCourseService.js:818  |
| <code>adminDeleteCourse()</code>             | Emergency course deletion. Removes: 1. Lessons 2. Modules 3. Course document 4. Course Storage files Teacher is notified BEFORE deletion.                                                                 | src/services/AdminCourseService.js:875  |
| <code>deleteCourseStorage()</code>           | Removes everything under: courses/{courseId} and: courseThumbnails/{courseId}                                                                                                                             | src/services/AdminCourseService.js:1045 |
| <code>deleteStorageFolder()</code>           | RECURSIVE STORAGE DELETE                                                                                                                                                                                  | src/services/AdminCourseService.js:1074 |
| <code>getCourseStats()</code>                | ADMIN - GET COURSE COUNTS                                                                                                                                                                                 | src/services/AdminCourseService.js:1110 |

| Function                              | Responsibility             | Source                                  |
|---------------------------------------|----------------------------|-----------------------------------------|
| <code>getCoursesByInstructor()</code> | FIND COURSES BY INSTRUCTOR | src/services/AdminCourseService.js:1178 |

## Application services - AssignmentService

| Function                                       | Responsibility                                                                                                         | Source                                |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <code>createAssignment(assignmentData)</code>  | CREATE ASSIGNMENT                                                                                                      | src/services/AssignmentService.js:25  |
| <code>getAssignment(assignmentId)</code>       | GET SINGLE ASSIGNMENT                                                                                                  | src/services/AssignmentService.js:49  |
| <code>publishAssignment(assignmentId)</code>   | PUBLISH ASSIGNMENT                                                                                                     | src/services/AssignmentService.js:132 |
| <code>unpublishAssignment(assignmentId)</code> | UNPUBLISH ASSIGNMENT                                                                                                   | src/services/AssignmentService.js:156 |
| <code>deleteAssignment(assignmentId)</code>    | DELETE ASSIGNMENT                                                                                                      | src/services/AssignmentService.js:179 |
| <code>getMyAssignments(courseIds = [])</code>  | GET MY ASSIGNMENTS courseIds = array of course IDs the student is enrolled in Only published assignments are returned. | src/services/AssignmentService.js:428 |

## Application services - AttendanceService

| Function                                              | Responsibility                                                                                                                                                                                  | Source                                |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <code>requireUser()</code>                            | Validates user for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                 | src/services/AttendanceService.js:25  |
| <code>getAttendanceId(sessionId, studentId)</code>    | Reads attendance id for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                            | src/services/AttendanceService.js:41  |
| <code>markCourseEngagementAttendance(courseId)</code> | Marks course engagement attendance for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                             | src/services/AttendanceService.js:49  |
| <code>timestampToMillis(timestamp)</code>             | Implements timestamp to millis for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                 | src/services/AttendanceService.js:70  |
| <code>sortNewestFirst(records)</code>                 | Implements sort newest first for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                   | src/services/AttendanceService.js:88  |
| <code>sortOldestFirst(records)</code>                 | Implements sort oldest first for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                   | src/services/AttendanceService.js:95  |
| <code>getRecordTime(record)</code>                    | Reads record time for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                              | src/services/AttendanceService.js:102 |
| <code>getTodayDate()</code>                           | Reads today date for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                               | src/services/AttendanceService.js:115 |
| <code>createAttendanceSession({})</code>              | Teacher/Admin creates an attendance session. Example: { courseId: "course123", teacherId: "teacher123", title: "Full Stack Development - Class", date: "2026-09-14" } Session starts as "open". | src/services/AttendanceService.js:139 |

| Function                                     | Responsibility                                                                                                                                                                                                                       | Source                                |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <code>getAttendanceSession()</code>          | Reads attendance session for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                            | src/services/AttendanceService.js:187 |
| <code>getCourseAttendanceSessions()</code>   | Reads course attendance sessions for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                    | src/services/AttendanceService.js:218 |
| <code>getOpenAttendanceSession()</code>      | Returns the currently open session for a course. We fetch course sessions and filter locally to avoid requiring another composite index.                                                                                             | src/services/AttendanceService.js:284 |
| <code>closeAttendanceSession()</code>        | Closes attendance session for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                           | src/services/AttendanceService.js:317 |
| <code>reopenAttendanceSession()</code>       | Useful for teacher/admin corrections.                                                                                                                                                                                                | src/services/AttendanceService.js:359 |
| <code>deleteAttendanceSession()</code>       | Admin/teacher can use this later according to Supabase database security rules. IMPORTANT: Deleting a session does NOT automatically delete attendance records belonging to it. That should be handled carefully at the admin level. | src/services/AttendanceService.js:409 |
| <code>markAttendance({})</code>              | Marks attendance for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                    | src/services/AttendanceService.js:441 |
| <code>getMySessionAttendance()</code>        | Reads my session attendance for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                         | src/services/AttendanceService.js:583 |
| <code>getMyAttendance()</code>               | Reads my attendance for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                 | src/services/AttendanceService.js:624 |
| <code>getCourseAttendance()</code>           | Reads course attendance for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                             | src/services/AttendanceService.js:659 |
| <code>getSessionAttendance()</code>          | Reads session attendance for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                            | src/services/AttendanceService.js:700 |
| <code>updateAttendance()</code>              | Used later by Teacher/Admin. Example: <code>updateAttendance("session_student", "absent")</code>                                                                                                                                     | src/services/AttendanceService.js:745 |
| <code>deleteAttendance()</code>              | Deletes attendance for the Attendance service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                  | src/services/AttendanceService.js:800 |
| <code>calculateAttendancePercentage()</code> | Calculate attendance percentage. Present = present Late = counted separately Absent = absent By default, percentage is: <code>present / total sessions</code>                                                                        | src/services/AttendanceService.js:846 |
| <code>getAttendanceSummary()</code>          | Returns complete attendance statistics. Example: <code>{ total: 30, present: 26, late: 2, absent: 2, percentage: 87 }</code>                                                                                                         | src/services/AttendanceService.js:886 |

## Application services - BatchService

| Function                                       | Responsibility                                                                                                                                                 | Source                           |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <code>requireUser()</code>                     | Validates user for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                     | src/services/BatchService.js:61  |
| <code>batchFromDoc(snapshot)</code>            | Implements batch from doc for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.          | src/services/BatchService.js:71  |
| <code>attendanceFromDoc(snapshot)</code>       | Implements attendance from doc for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.     | src/services/BatchService.js:78  |
| <code>normalizeStudentIds(value)</code>        | Normalizes student ids for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/BatchService.js:85  |
| <code>normalizeSchedule(schedule)</code>       | Normalizes schedule for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                | src/services/BatchService.js:89  |
| <code>getAttendanceDocId(batchId, date)</code> | Reads attendance doc id for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.            | src/services/BatchService.js:102 |
| <code>isValidDateString(date)</code>           | Implements is valid date string for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.    | src/services/BatchService.js:106 |
| <code>assertMode(mode)</code>                  | Enforces mode for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                      | src/services/BatchService.js:110 |
| <code>assertAttendanceRecords(records)</code>  | Enforces attendance records for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.        | src/services/BatchService.js:116 |
| <code>sortBatches(batches)</code>              | Implements sort batches for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.            | src/services/BatchService.js:134 |
| <code>sortAttendanceByDate(records)</code>     | Implements sort attendance by date for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/BatchService.js:142 |
| <code>weekdayIndex(day)</code>                 | Implements weekday index for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/services/BatchService.js:148 |
| <code>getNextScheduledSession(schedule)</code> | Reads next scheduled session for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.       | src/services/BatchService.js:152 |
| <code>getStudentProfile(studentId)</code>      | Reads student profile for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.              | src/services/BatchService.js:194 |
| <code>hydrateStudents(studentIds)</code>       | Implements hydrate students for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.        | src/services/BatchService.js:248 |
| <code>getBatch(batchId)</code>                 | Reads batch for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                        | src/services/BatchService.js:262 |
| <code>createBatch()</code>                     | Creates batch for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                      | src/services/BatchService.js:283 |

| Function                                                                           | Responsibility                                                                                                                                                                                          | Source                           |
|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <code>editBatch(batchId, updates = {})</code>                                      | Implements edit batch for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                       | src/services/BatchService.js:350 |
| <code>archiveBatch(batchId)</code>                                                 | Implements archive batch for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                    | src/services/BatchService.js:405 |
| <code>deleteBatch(batchId)</code>                                                  | Deletes batch for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                               | src/services/BatchService.js:434 |
| <code>getBatchesForTeacher(teacherId)</code>                                       | Reads batches for teacher for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                   | src/services/BatchService.js:465 |
| <code>getBatchesForCourse(courseId)</code>                                         | Reads batches for course for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                    | src/services/BatchService.js:494 |
| <code>getEligibleStudents(courseId, batchId)</code>                                | Reads eligible students for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                     | src/services/BatchService.js:517 |
| <code>searchBatchStudents(courseId, batchId, searchText = "")</code>               | Secure server-side directory search for a teacher's own batch. This avoids exposing every profile to the browser while still supporting type-ahead by the first character of a student's name or email. | src/services/BatchService.js:564 |
| <code>changeBatchStudent(action, batchId, studentUid, destinationId = null)</code> | Implements change batch student for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                             | src/services/BatchService.js:588 |
| <code>addStudentToBatch(batchId, studentUid)</code>                                | Implements add student to batch for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                             | src/services/BatchService.js:596 |
| <code>removeStudentFromBatch(batchId, studentUid)</code>                           | Removes student from batch for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                  | src/services/BatchService.js:597 |
| <code>shiftStudentBatch(oldBatchId, newBatchId, studentUid)</code>                 | Implements shift student batch for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                              | src/services/BatchService.js:598 |
| <code>markAttendance()</code>                                                      | Marks attendance for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                            | src/services/BatchService.js:600 |
| <code>getAttendanceForBatch(batchId)</code>                                        | Reads attendance for batch for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                  | src/services/BatchService.js:690 |
| <code>getAttendanceForDate(batchId, date)</code>                                   | Reads attendance for date for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                   | src/services/BatchService.js:714 |
| <code>getStudentAttendanceStats()</code>                                           | Reads student attendance stats for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                              | src/services/BatchService.js:743 |
| <code>getBatchRoster(batchId)</code>                                               | Reads batch roster for the Batch service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                          | src/services/BatchService.js:794 |

## Application services - Certificatepdf

| Function                                  | Responsibility                                                                                                                                         | Source                             |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| <code>buildCertificatePdf({})</code>      | Draws the certificate and returns the jsPDF instance so callers can .save(), .output("blob"), .output("dataurlstring"), etc.                           | src/services/Certificatepdf.js:18  |
| <code>downloadCertificatePdf(data)</code> | Downloads certificate pdf for the Certificatepdf area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/Certificatepdf.js:113 |

## Application services - Certificates

| Function                                         | Responsibility                                                                                                                                                                                                                                                                                                                             | Source                          |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| <code>generateCertificateId(courseId)</code>     | Implements generate certificate id for the Certificates area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                              | src/services/Certificates.js:10 |
| <code>listenToCertificates(uid, callback)</code> | Live list of certificates this user has actually earned, newest first.                                                                                                                                                                                                                                                                     | src/services/Certificates.js:17 |
| <code>watchForCourseCompletion()</code>          | Watches this user's enrollments. Any enrollment at progress === 100 that doesn't already have a matching certificate gets one issued automatically. Call once (e.g. in a top-level effect) and keep the unsubscribe around. Certificates are issued atomically by lms_complete_lesson in PostgreSQL. Retained for dashboard callers; no br | src/services/Certificates.js:37 |

## Application services - CommunicationService

| Function                                     | Responsibility                                                                                                                                                                            | Source                                                |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| <code>requireAuthUser()</code>               | Require logged-in Supabase user.                                                                                                                                                          | <code>src/services/CommunicationService.js:66</code>  |
| <code>documentToObject(snapshot)</code>      | Convert Supabase database document into normal object.                                                                                                                                    | <code>src/services/CommunicationService.js:79</code>  |
| <code>getTimestampValue(value)</code>        | Convert Supabase database Timestamp / Date / number into milliseconds.                                                                                                                    | <code>src/services/CommunicationService.js:90</code>  |
| <code>sortNewestFirst(items = [])</code>     | Sort newest first. Client-side sorting intentionally avoids Supabase database composite index requirements.                                                                               | <code>src/services/CommunicationService.js:116</code> |
| <code>removeDuplicates(items = [])</code>    | Remove duplicate documents.                                                                                                                                                               | <code>src/services/CommunicationService.js:127</code> |
| <code>cleanString(value)</code>              | Safely trim a string.                                                                                                                                                                     | <code>src/services/CommunicationService.js:142</code> |
| <code>getCurrentUserRole()</code>            | Get current user's role. Priority: 1. Supabase custom claims 2. <code>users/{uid}.role</code>                                                                                             | <code>src/services/CommunicationService.js:159</code> |
| <code>requireAdmin()</code>                  | Require admin.                                                                                                                                                                            | <code>src/services/CommunicationService.js:244</code> |
| <code>requireTeacher()</code>                | Require teacher.                                                                                                                                                                          | <code>src/services/CommunicationService.js:260</code> |
| <code>getCourse(courseId)</code>             | Get course.                                                                                                                                                                               | <code>src/services/CommunicationService.js:280</code> |
| <code>isCourseOwner()</code>                 | Check whether user owns course.                                                                                                                                                           | <code>src/services/CommunicationService.js:299</code> |
| <code>getMyActiveCourseIds()</code>          | Get active course IDs for current student. Enrollment structure: <code>enrollments/{uid}_{courseId}</code> with: <code>uid courseId status: "active"</code>                               | <code>src/services/CommunicationService.js:331</code> |
| <code>getMyTeacherCourses()</code>           | Get all courses owned by current teacher.                                                                                                                                                 | <code>src/services/CommunicationService.js:375</code> |
| <code>canManageAnnouncement()</code>         | Check whether current user can manage a specific announcement. ADMIN: - Everything TEACHER: - Own course announcement - Own announcement only                                             | <code>src/services/CommunicationService.js:418</code> |
| <code>getAnnouncements({})</code>            | ADMIN Get all announcements. Includes: - drafts - published - global - course announcements                                                                                               | <code>src/services/CommunicationService.js:474</code> |
| <code>getGlobalAnnouncements({})</code>      | Get published global announcements. ACCESS: ADMIN TEACHER STUDENT This is the important change: TEACHERS ARE NOW ALLOWED TO READ PUBLISHED GLOBAL ANNOUNCEMENTS.                          | <code>src/services/CommunicationService.js:518</code> |
| <code>getCourseAnnouncements()</code>        | Get published announcements for one course. Access is ultimately controlled by Supabase database rules. Student: - must be enrolled Teacher: - must own course Admin: - allowed           | <code>src/services/CommunicationService.js:595</code> |
| <code>getAccessibleAnnouncements()</code>    | Get announcements accessible to current student. Returns: 1. Published global announcements 2. Published announcements for active courses                                                 | <code>src/services/CommunicationService.js:661</code> |
| <code>getMyTeacherAnnouncements({})</code>   | Get announcements visible to teacher. Teacher receives: 1. Published global announcements 2. Published announcements for their own courses This is what the teacher dashboard should use. | <code>src/services/CommunicationService.js:781</code> |
| <code>getTeacherOnlyAnnouncements({})</code> | Get published announcements intended only for teacher accounts.                                                                                                                           | <code>src/services/CommunicationService.js:913</code> |

| Function                                       | Responsibility                                                                                                                                                                                                                                                                                                                                       | Source                                                 |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <code>getMyCreatedAnnouncements()</code>       | Get announcements created by current teacher. IMPORTANT: We query through courses owned by the teacher instead of: <code>where("authorId", "=", uid)</code> because Supabase database security rules validate teacher access through course ownership. This function intentionally includes drafts because teachers need to manage their own drafts. | <code>src/services/CommunicationService.js:950</code>  |
| <code>getTeacherActivityNotifications()</code> | Build a notification feed from activity in the teacher's courses. These are derived from the source collections, so no duplicate notification documents are needed.                                                                                                                                                                                  | <code>src/services/CommunicationService.js:1050</code> |
| <code>createAnnouncement({})</code>            | ADMIN: - Global - Course TEACHER: - Course only - Own course only - Published course only                                                                                                                                                                                                                                                            | <code>src/services/CommunicationService.js:1153</code> |
| <code>updateAnnouncement()</code>              | Updates announcement for the Communication service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                             | <code>src/services/CommunicationService.js:1364</code> |
| <code>deleteAnnouncement()</code>              | Deletes announcement for the Communication service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                             | <code>src/services/CommunicationService.js:1543</code> |
| <code>toggleAnnouncementPin()</code>           | Implements toggle announcement pin for the Communication service area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                               | <code>src/services/CommunicationService.js:1599</code> |
| <code>getCourseDiscussions()</code>            | Get discussions for course.                                                                                                                                                                                                                                                                                                                          | <code>src/services/CommunicationService.js:1666</code> |
| <code>getDiscussion()</code>                   | Get one discussion.                                                                                                                                                                                                                                                                                                                                  | <code>src/services/CommunicationService.js:1711</code> |
| <code>createDiscussion({})</code>              | Create discussion.                                                                                                                                                                                                                                                                                                                                   | <code>src/services/CommunicationService.js:1743</code> |
| <code>updateDiscussion()</code>                | Update discussion.                                                                                                                                                                                                                                                                                                                                   | <code>src/services/CommunicationService.js:1832</code> |
| <code>deleteDiscussion()</code>                | Delete discussion.                                                                                                                                                                                                                                                                                                                                   | <code>src/services/CommunicationService.js:1921</code> |
| <code>resolveDiscussion()</code>               | Resolve discussion.                                                                                                                                                                                                                                                                                                                                  | <code>src/services/CommunicationService.js:1946</code> |
| <code>getDiscussionReplies()</code>            | Get replies.                                                                                                                                                                                                                                                                                                                                         | <code>src/services/CommunicationService.js:1982</code> |
| <code>createDiscussionReply()</code>           | Create reply.                                                                                                                                                                                                                                                                                                                                        | <code>src/services/CommunicationService.js:2022</code> |
| <code>deleteDiscussionReply()</code>           | Delete reply.                                                                                                                                                                                                                                                                                                                                        | <code>src/services/CommunicationService.js:2089</code> |
| <code>getMyNotifications({})</code>            | Get current user's notifications. Canonical field: <code>recipientId</code>                                                                                                                                                                                                                                                                          | <code>src/services/CommunicationService.js:2128</code> |
| <code>getAdminActivityNotifications({})</code> | Build the admin activity feed without Cloud Functions. This reads source collections directly, so it works on the Spark plan.                                                                                                                                                                                                                        | <code>src/services/CommunicationService.js:2166</code> |
| <code>getUnreadNotificationCount()</code>      | Get unread notification count.                                                                                                                                                                                                                                                                                                                       | <code>src/services/CommunicationService.js:2283</code> |
| <code>markNotificationRead()</code>            | Mark one notification as read.                                                                                                                                                                                                                                                                                                                       | <code>src/services/CommunicationService.js:2318</code> |
| <code>markAllNotificationsRead()</code>        | Mark all notifications as read.                                                                                                                                                                                                                                                                                                                      | <code>src/services/CommunicationService.js:2350</code> |
| <code>deleteNotification()</code>              | Delete notification.                                                                                                                                                                                                                                                                                                                                 | <code>src/services/CommunicationService.js:2413</code> |

| Function                                  | Responsibility                                                                                                                                                | Source                                    |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <code>createNotification({})</code>       | Create notification. Current security architecture: Admin / Cloud Functions create notifications. Students and teachers do not directly create notifications. | src/services/CommunicationService.js:2449 |
| <code>subscribeToMyNotifications()</code> | Subscribe to current user's notifications. No orderBy(). Client-side sorting avoids composite indexes.                                                        | src/services/CommunicationService.js:2544 |

## Application services - CourseCommunityService

| Function                                           | Responsibility                                                                                                                                                             | Source                                    |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <code>validateWhatsAppInvite(value)</code>         | Implements validate whats app invite for the CourseCommunity service area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/CourseCommunityService.js:6  |
| <code>requireUser()</code>                         | Validates user for the CourseCommunity service area. Source behavior and authorization are enforced by the surrounding module and database policies.                       | src/services/CourseCommunityService.js:11 |
| <code>getCourseCommunity(courseId)</code>          | Reads course community for the CourseCommunity service area. Source behavior and authorization are enforced by the surrounding module and database policies.               | src/services/CourseCommunityService.js:15 |
| <code>getStudentCourseCommunity(courseId)</code>   | Reads student course community for the CourseCommunity service area. Source behavior and authorization are enforced by the surrounding module and database policies.       | src/services/CourseCommunityService.js:26 |
| <code>saveCourseCommunity(courseId, values)</code> | Persists course community for the CourseCommunity service area. Source behavior and authorization are enforced by the surrounding module and database policies.            | src/services/CourseCommunityService.js:37 |
| <code>removeCourseCommunity(courseId)</code>       | Removes course community for the CourseCommunity service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/CourseCommunityService.js:60 |
| <code>getCommunityJoinClickCount(courseId)</code>  | Reads community join click count for the CourseCommunity service area. Source behavior and authorization are enforced by the surrounding module and database policies.     | src/services/CourseCommunityService.js:69 |
| <code>openCourseWhatsAppGroup(courseId)</code>     | Opens course whats app group for the CourseCommunity service area. Source behavior and authorization are enforced by the surrounding module and database policies.         | src/services/CourseCommunityService.js:76 |

## Application services - CourseContent

| Function                             | Responsibility                                                                                                                                          | Source                             |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| <code>isYouTubeLink(value)</code>    | Implements is you tube link for the CourseContent area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/CourseContent.js:56   |
| <code>CourseContent()</code>         | Implements course content for the CourseContent area. Source behavior and authorization are enforced by the surrounding module and database policies.   | src/services/CourseContent.js:68   |
| <code>LessonRow({})</code>           | LESSON ROW                                                                                                                                              | src/services/CourseContent.js:1746 |
| <code>Modal({})</code>               | MODAL                                                                                                                                                   | src/services/CourseContent.js:1877 |
| <code>LessonTypeCard({})</code>      | TYPE CARD                                                                                                                                               | src/services/CourseContent.js:1917 |
| <code>StatusBadge({ status })</code> | STATUS                                                                                                                                                  | src/services/CourseContent.js:1972 |

| Function        | Responsibility | Source                                  |
|-----------------|----------------|-----------------------------------------|
| IconButton({})  | ICON BUTTON    | src/services/CourseContent.js<br>x:2018 |
| LoadingScreen() | LOADING        | src/services/CourseContent.js<br>x:2047 |

## Application services - CourseContentService

| Function                                              | Responsibility                                                                                                                                                 | Source                                   |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <code>requireUser()</code>                            | Validates user for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/CourseContentService.js:29  |
| <code>requireCourseOwner(courseId)</code>             | Validates course owner for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.     | src/services/CourseContentService.js:43  |
| <code>getModulesCollection(courseId)</code>           | Reads modules collection for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.   | src/services/CourseContentService.js:93  |
| <code>getModuleReference(courseId, moduleId)</code>   | Reads module reference for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.     | src/services/CourseContentService.js:102 |
| <code>getLessonsCollection(courseId, moduleId)</code> | Reads lessons collection for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.   | src/services/CourseContentService.js:112 |
| <code>getLessonReference()</code>                     | Reads lesson reference for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.     | src/services/CourseContentService.js:123 |
| <code>getCourseModules(courseId)</code>               | Reads course modules for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.       | src/services/CourseContentService.js:143 |
| <code>getModuleLessons()</code>                       | Reads module lessons for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.       | src/services/CourseContentService.js:163 |
| <code>getCourseContent(courseId)</code>               | Reads course content for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.       | src/services/CourseContentService.js:190 |
| <code>createModule()</code>                           | Creates module for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/CourseContentService.js:216 |
| <code>updateModule()</code>                           | Updates module for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/CourseContentService.js:260 |
| <code>deleteModule()</code>                           | Deletes module for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/CourseContentService.js:309 |
| <code>reorderModules()</code>                         | Implements reorder modules for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/CourseContentService.js:368 |
| <code>createLesson()</code>                           | Creates lesson for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/CourseContentService.js:400 |
| <code>updateLesson()</code>                           | Updates lesson for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/CourseContentService.js:501 |
| <code>deleteLesson()</code>                           | Deletes lesson for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/CourseContentService.js:576 |
| <code>reorderLessons()</code>                         | Implements reorder lessons for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/CourseContentService.js:632 |

| Function                                                   | Responsibility                                                                                                                                            | Source                                   |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <code>setLessonPublished()</code>                          | Sets lesson published for the CourseContent service area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/CourseContentService.js:673 |
| <code>listenToCourses(callback)</code>                     | Subscribes to the current instructor's own courses in real time.                                                                                          | src/services/CourseContentService.js:713 |
| <code>listenToModules(courseId, callback)</code>           | Subscribes to a course's modules in real time, ordered by `order`.                                                                                        | src/services/CourseContentService.js:730 |
| <code>listenToLessons(courseId, moduleId, callback)</code> | Subscribes to a module's lessons in real time, ordered by `order`.                                                                                        | src/services/CourseContentService.js:747 |

## Application services - CourseService

| Function                                                           | Responsibility                                                                                                                                            | Source                            |
|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| <code>slugify(title = "")</code>                                   | Implements slugify for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/services/CourseService.js:42  |
| <code>sortCourses(a, b)</code>                                     | Implements sort courses for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.      | src/services/CourseService.js:51  |
| <code>requireAuthUser()</code>                                     | Validates auth user for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.          | src/services/CourseService.js:62  |
| <code>friendlyDbError(error, fallbackMessage)</code>               | Implements friendly db error for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/CourseService.js:80  |
| <code>requireCourseOwner(courseId)</code>                          | Validates course owner for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.       | src/services/CourseService.js:95  |
| <code>groupByCategory(courses = [])</code>                         | Implements group by category for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/CourseService.js:123 |
| <code>getMyCourses(teacherId)</code>                               | Reads my courses for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/CourseService.js:145 |
| <code>useMyCourses(teacherIdParam, { enabled = true } = {})</code> | Implements use my courses for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.    | src/services/CourseService.js:174 |
| <code>getCourseById(courseId)</code>                               | Reads course by id for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/services/CourseService.js:192 |
| <code>createCourse(courseData = {})</code>                         | Creates course for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.               | src/services/CourseService.js:224 |
| <code>updateCourse(courseId, courseData = {})</code>               | Updates course for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.               | src/services/CourseService.js:284 |
| <code>deleteCourse(courseId)</code>                                | Deletes course for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.               | src/services/CourseService.js:372 |
| <code>publishCourse(courseId)</code>                               | Implements publish course for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.    | src/services/CourseService.js:409 |

| Function                                                       | Responsibility                                                                                                                                                            | Source                            |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| <code>unpublishCourse(courseId)</code>                         | Implements unpublish course for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.                  | src/services/CourseService.js:439 |
| <code>usePublishedCourses(type)</code>                         | Implements use published courses for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/CourseService.js:463 |
| <code>usePublishedCoursesByCategory(type)</code>               | Implements use published courses by category for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/CourseService.js:474 |
| <code>getPublishedCourseByIdOrSlug(idOrSlug)</code>            | Reads published course by id or slug for the Course service area. Source behavior and authorization are enforced by the surrounding module and database policies.         | src/services/CourseService.js:484 |
| <code>uploadCourseThumbnail(courseId, file, onProgress)</code> | Thumbnails are uploaded to the public Supabase image bucket.                                                                                                              | src/services/CourseService.js:513 |
| <code>deleteCourseThumbnail(path)</code>                       | Remove the previous thumbnail after replacement.                                                                                                                          | src/services/CourseService.js:539 |
| <code>safeDeleteStorageFile(path)</code>                       | Private and public storage paths are routed to their bucket.                                                                                                              | src/services/CourseService.js:545 |

## Application services - EnrollmentService

| Function                                                         | Responsibility                                                                                                                                          | Source                                |
|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <code>requireUser()</code>                                       | Validates user for the Enrollment service area. Source behavior and authorization are enforced by the surrounding module and database policies.         | src/services/EnrollmentService.js:29  |
| <code>getEnrollmentId(uid, courseId)</code>                      | Reads enrollment id for the Enrollment service area. Source behavior and authorization are enforced by the surrounding module and database policies.    | src/services/EnrollmentService.js:39  |
| <code>getOwnedEnrollment()</code>                                | Loads an enrollment document and makes sure it belongs to the logged-in student. Optionally requires the enrollment to be active.                       | src/services/EnrollmentService.js:47  |
| <code>getMyEnrollments()</code>                                  | Reads my enrollments for the Enrollment service area. Source behavior and authorization are enforced by the surrounding module and database policies.   | src/services/EnrollmentService.js:81  |
| <code>getEnrollment(uid, courseId)</code>                        | Reads enrollment for the Enrollment service area. Source behavior and authorization are enforced by the surrounding module and database policies.       | src/services/EnrollmentService.js:113 |
| <code>getEnrollmentById(enrollmentId)</code>                     | Reads enrollment by id for the Enrollment service area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/EnrollmentService.js:134 |
| <code>enrollStudent({ courseId, paymentStatus = "free" })</code> | Enrolls student for the Enrollment service area. Source behavior and authorization are enforced by the surrounding module and database policies.        | src/services/EnrollmentService.js:151 |
| <code>markLessonComplete(enrollmentId, lessonId)</code>          | Marks lesson complete for the Enrollment service area. Source behavior and authorization are enforced by the surrounding module and database policies.  | src/services/EnrollmentService.js:176 |
| <code>updateLastLesson(enrollmentId, lessonId)</code>            | Updates last lesson for the Enrollment service area. Source behavior and authorization are enforced by the surrounding module and database policies.    | src/services/EnrollmentService.js:197 |
| <code>getLessonWatchProgress(enrollmentId)</code>                | Returns every saved watch row for this enrollment: [{ lessonId, watchedSeconds, videoEnded }]                                                           | src/services/EnrollmentService.js:232 |
| <code>recordLessonWatch()</code>                                 | Adds watched seconds for a lesson and/or flags the video as ended. Returns { watchedSeconds, videoEnded } for that lesson.                              | src/services/EnrollmentService.js:253 |

## Application services - Payments

| Function                                                                                     | Responsibility                                                                                                                                       | Source                      |
|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| <code>invoke(action, payload)</code>                                                         | Implements invoke for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/Payments.js:2  |
| <code>createPaymentOrder(courseId, paymentMode = "one_time", installmentCount = null)</code> | Creates payment order for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies.         | src/services/Payments.js:12 |
| <code>verifyPayment({ orderId, paymentId, signature })</code>                                | Verifies payment for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies.              | src/services/Payments.js:13 |
| <code>getInstallmentPlan(courseId)</code>                                                    | Reads installment plan for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies.        | src/services/Payments.js:14 |
| <code>syncEmiReminders()</code>                                                              | Synchronizes emi reminders for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies.    | src/services/Payments.js:15 |
| <code>refundPayment(paymentId, reason)</code>                                                | Refunds payment for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies.               | src/services/Payments.js:16 |
| <code>createEmiInstallmentOrder(installmentId)</code>                                        | Creates emi installment order for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/Payments.js:17 |

## Application services - StorageService

| Function                                                           | Responsibility                                                                                                                                              | Source                             |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| <code>requireUser()</code>                                         | Validates user for the Storage service area. Source behavior and authorization are enforced by the surrounding module and database policies.                | src/services/StorageService.js:14  |
| <code>validateFile(file, allowedTypes = [], maxSizeMB = 50)</code> | Implements validate file for the Storage service area. Source behavior and authorization are enforced by the surrounding module and database policies.      | src/services/StorageService.js:28  |
| <code>createFileName(file)</code>                                  | Creates file name for the Storage service area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/services/StorageService.js:55  |
| <code>uploadFile()</code>                                          | Uploads file for the Storage service area. Source behavior and authorization are enforced by the surrounding module and database policies.                  | src/services/StorageService.js:71  |
| <code>uploadLessonVideo()</code>                                   | Uploads lesson video for the Storage service area. Source behavior and authorization are enforced by the surrounding module and database policies.          | src/services/StorageService.js:230 |
| <code>uploadLessonThumbnail()</code>                               | Uploads lesson thumbnail for the Storage service area. Source behavior and authorization are enforced by the surrounding module and database policies.      | src/services/StorageService.js:267 |
| <code>uploadLessonResource()</code>                                | Uploads lesson resource for the Storage service area. Source behavior and authorization are enforced by the surrounding module and database policies.       | src/services/StorageService.js:303 |
| <code>uploadAssignmentSubmission()</code>                          | Uploads assignment submission for the Storage service area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/services/StorageService.js:347 |
| <code>deleteStorageFile(storagePath)</code>                        | Deletes storage file for the Storage service area. Source behavior and authorization are enforced by the surrounding module and database policies.          | src/services/StorageService.js:398 |

## Application services - UserProfile

| Function                         | Responsibility                                                              | Source                        |
|----------------------------------|-----------------------------------------------------------------------------|-------------------------------|
| <code>ensureUserDoc(user)</code> | The auth.users trigger creates profiles; clients cannot choose their roles. | src/services/UserProfile.js:3 |

## Authentication and route guards

| Function                                     | Responsibility                                                                                                                                  | Source                        |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| <code>AccessGate({ roles, children })</code> | Implements access gate for the AccessGate area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/auth/AccessGate.jsx:6     |
| <code>RequireAdmin({ children })</code>      | Validates admin for the RequireAdmin area. Source behavior and authorization are enforced by the surrounding module and database policies.      | src/auth/RequireAdmin.jsx:2   |
| <code>RequireAuth({ children })</code>       | Validates auth for the RequireAuth area. Source behavior and authorization are enforced by the surrounding module and database policies.        | src/auth/RequireAuth.jsx:2    |
| <code>RequireStudent({ children })</code>    | Validates student for the RequireStudent area. Source behavior and authorization are enforced by the surrounding module and database policies.  | src/auth/RequireStudent.jsx:3 |
| <code>RequireTeacher({ children })</code>    | Validates teacher for the RequireTeacher area. Source behavior and authorization are enforced by the surrounding module and database policies.  | src/auth/RequireTeacher.jsx:2 |

## Edge Functions - \_shared

| Function                                                                                 | Responsibility                                                                                                                                    | Source                                    |
|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <code>payableAmount(course)</code>                                                       | Implements payable amount for the payments area. Source behavior and authorization are enforced by the surrounding module and database policies.  | supabase/functions/_shared/payments.js:1  |
| <code>validSignature(secret, message, signature)</code>                                  | Implements valid signature for the payments area. Source behavior and authorization are enforced by the surrounding module and database policies. | supabase/functions/_shared/payments.js:8  |
| <code>assertCapturedPayment(payment, order)</code>                                       | Enforces captured payment for the payments area. Source behavior and authorization are enforced by the surrounding module and database policies.  | supabase/functions/_shared/payments.js:15 |
| <code>sendPaymentReceipt({ payment, courseTitle, recipientEmail, recipientName })</code> | Sends payment receipt for the receipts area. Source behavior and authorization are enforced by the surrounding module and database policies.      | supabase/functions/_shared/receipts.js:5  |

## Edge Functions - lms-payment-webhook

| Function                              | Responsibility                                                                                                                     | Source                                            |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| <code>updateRefund(event: any)</code> | Updates refund for the index area. Source behavior and authorization are enforced by the surrounding module and database policies. | supabase/functions/lms-payment-webhook/index.ts:7 |

## Edge Functions - lms-payments

| Function                                                                                                                     | Responsibility                                                                                                                                        | Source                                      |
|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| <code>applyProcessedRefund(order : Record&lt;string, any&gt;, refund: Record&lt;string, any&gt;, requestedBy: string)</code> | Implements apply processed refund for the index area. Source behavior and authorization are enforced by the surrounding module and database policies. | supabase/functions/lms-payments/index.ts:11 |
| <code>durationMonths(value: unknown)</code>                                                                                  | Implements duration months for the index area. Source behavior and authorization are enforced by the surrounding module and database policies.        | supabase/functions/lms-payments/index.ts:34 |
| <code>razorpay(path: string, body?: Record&lt;string, unknown&gt;)</code>                                                    | Implements razorpay for the index area. Source behavior and authorization are enforced by the surrounding module and database policies.               | supabase/functions/lms-payments/index.ts:46 |

## Primary page and UI component functions

| Function                                                                            | Responsibility                                                                                                                                                                                                                                                                                                                                                      | Source                          |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| <code>AboutSection()</code>                                                         | Implements about section for the About area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                                        | src/pages/About.jsx:157         |
| <code>AuthCallback()</code>                                                         | AuthCallback - where Supabase sends the browser back to after Google OAuth. Add this route in App.jsx: <code>&lt;Route path="/auth/callback" element={}&lt;AuthCallback /&gt; /&gt;</code> Also add the same URL (your-domain/auth/callback) to Supabase's Authentication -> URL Configuration -> Redirect URLs allow-list, for both localhost (dev) and your produ | src/pages/AuthCallback.jsx:16   |
| <code>ContactSection()</code>                                                       | Implements contact section for the Contact area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                                    | src/pages/Contact.jsx:151       |
| <code>CourseDetails()</code>                                                        | Implements course details for the CourseDetails area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                               | src/pages/CourseDetails.jsx:133 |
| <code>ForgotPassword()</code>                                                       | Implements forgot password for the ForgotPassword area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                             | src/pages/ForgotPassword.jsx:7  |
| <code>Home()</code>                                                                 | Implements home for the Home area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                                                  | src/pages/Home.jsx:31           |
| <code>LegalPage({ eyebrow, title, updated = "25 September 2026", children })</code> | Implements legal page for the LegalPage area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                                       | src/pages/LegalPage.jsx:3       |
| <code>DotGrid({ className = "", dot = "fill-white/25" })</code>                     | Implements dot grid for the Login area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                                             | src/pages/Login.jsx:16          |
| <code>LoginForm()</code>                                                            | Implements login form for the Login area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                                           | src/pages/Login.jsx:77          |
| <code>PrivacyPolicy()</code>                                                        | Implements privacy policy for the PrivacyPolicy area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                               | src/pages/PrivacyPolicy.jsx:3   |
| <code>RefundPolicy()</code>                                                         | Refunds policy for the RefundPolicy area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                                           | src/pages/RefundPolicy.jsx:3    |
| <code>DotGrid({ className = "", dot = "fill-white/25" })</code>                     | Register - Creative Adhyayan Same Google sign-in handling as LoginForm - keep the two in sync. Supabase's OAuth is redirect-based (not a popup), so the actual post-Google routing happens in /auth/callback (AuthCallback.jsx), not in this component.                                                                                                             | src/pages/Register.jsx:28       |
| <code>RegisterForm()</code>                                                         | Implements register form for the Register area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                                     | src/pages/Register.jsx:78       |
| <code>ResetPassword()</code>                                                        | Implements reset password for the ResetPassword area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                               | src/pages/ResetPassword.jsx:5   |
| <code>ShortCourses()</code>                                                         | Implements short courses for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                                 | src/pages/ShortCourses.jsx:230  |
| <code>NewCourseLaunches({ launches })</code>                                        | Implements new course launches for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                                                                                                                                                                           | src/pages/ShortCourses.jsx:506  |

| Function                                       | Responsibility                                                                                                                                              | Source                           |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <code>LaunchCard({ course })</code>            | Implements launch card for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/pages/ShortCourses.jsx:608   |
| <code>ScrollToTopButton({ onClick })</code>    | Implements scroll to top button for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.  | src/pages/ShortCourses.jsx:685   |
| <code>PageHeader({ courseCount })</code>       | Implements page header for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/pages/ShortCourses.jsx:739   |
| <code>TrustPoint({ text })</code>              | Implements trust point for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/pages/ShortCourses.jsx:854   |
| <code>FilterBar({})</code>                     | Implements filter bar for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.            | src/pages/ShortCourses.jsx:870   |
| <code>CourseResultsHeader({})</code>           | Implements course results header for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/pages/ShortCourses.jsx:1099  |
| <code>CourseCard({ course })</code>            | Implements course card for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/pages/ShortCourses.jsx:1154  |
| <code>Price({ pricing })</code>                | Implements price for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.                 | src/pages/ShortCourses.jsx:1524  |
| <code>CourseGridSkeleton({ count = 6 })</code> | Implements course grid skeleton for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.  | src/pages/ShortCourses.jsx:1568  |
| <code>EmptyState({})</code>                    | Implements empty state for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/pages/ShortCourses.jsx:1623  |
| <code>ErrorState({ message })</code>           | Implements error state for the ShortCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/pages/ShortCourses.jsx:1708  |
| <code>Terms()</code>                           | Implements terms for the Terms area. Source behavior and authorization are enforced by the surrounding module and database policies.                        | src/pages/Terms.jsx:3            |
| <code>Assignments({})</code>                   | Implements assignments for the Assignments area. Source behavior and authorization are enforced by the surrounding module and database policies.            | src/student/Assignments.jsx:130  |
| <code>Attendance()</code>                      | Implements attendance for the Attendance area. Source behavior and authorization are enforced by the surrounding module and database policies.              | src/student/Attendance.jsx:87    |
| <code>Certificates()</code>                    | Implements certificates for the Certificates area. Source behavior and authorization are enforced by the surrounding module and database policies.          | src/student/Certificates.jsx:21  |
| <code>Dashboard()</code>                       | Implements dashboard for the Dashboard area. Source behavior and authorization are enforced by the surrounding module and database policies.                | src/student/Dashboard.jsx:416    |
| <code>LearnCourse()</code>                     | Implements learn course for the LearnCourse area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/student/LearnCourse.jsx:179  |
| <code>LockedContent()</code>                   | Implements locked content for the LearnCourse area. Source behavior and authorization are enforced by the surrounding module and database policies.         | src/student/LearnCourse.jsx:2013 |

| Function                                                                | Responsibility                                                                                                                                             | Source                           |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <code>SeekButton({ label, disabled = false, onClick, children })</code> | Implements seek button for the LearnCourse area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/student/LearnCourse.jsx:2047 |
| <code>MyCourses()</code>                                                | Implements my courses for the MyCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.              | src/student/MyCourses.jsx:94     |
| <code>StatCard({})</code>                                               | Implements stat card for the MyCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.               | src/student/MyCourses.jsx:703    |
| <code>PaymentDetails({ enrollment })</code>                             | Implements payment details for the MyCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.         | src/student/MyCourses.jsx:740    |
| <code>StudentCourseCard({})</code>                                      | Implements student course card for the MyCourses area. Source behavior and authorization are enforced by the surrounding module and database policies.     | src/student/MyCourses.jsx:832    |
| <code>StudentPayments()</code>                                          | Implements student payments for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies.         | src/student/Payments.jsx:19      |
| <code>Profile()</code>                                                  | Implements profile for the Profile area. Source behavior and authorization are enforced by the surrounding module and database policies.                   | src/student/Profile.jsx:29       |
| <code>ToastStack({ toasts, onDismiss })</code>                          | Implements toast stack for the Assignments area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/Teacher/Assignments.jsx:148  |
| <code>ConfirmModal({ confirmState, onCancel, onConfirm })</code>        | Implements confirm modal for the Assignments area. Source behavior and authorization are enforced by the surrounding module and database policies.         | src/Teacher/Assignments.jsx:203  |
| <code>TeacherAssignments()</code>                                       | Implements teacher assignments for the Assignments area. Source behavior and authorization are enforced by the surrounding module and database policies.   | src/Teacher/Assignments.jsx:252  |
| <code>Attendance()</code>                                               | Implements attendance for the Attendance area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/Teacher/Attendance.jsx:100   |
| <code>MobileAttendanceCard({})</code>                                   | Implements mobile attendance card for the Attendance area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/Teacher/Attendance.jsx:645   |
| <code>DesktopAttendanceRow({})</code>                                   | Implements desktop attendance row for the Attendance area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/Teacher/Attendance.jsx:774   |
| <code>LoadingState({})</code>                                           | Implements loading state for the Attendance area. Source behavior and authorization are enforced by the surrounding module and database policies.          | src/Teacher/Attendance.jsx:859   |
| <code>EmptyState({})</code>                                             | Implements empty state for the Attendance area. Source behavior and authorization are enforced by the surrounding module and database policies.            | src/Teacher/Attendance.jsx:888   |
| <code>SummaryCard({})</code>                                            | Implements summary card for the Attendance area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/Teacher/Attendance.jsx:920   |
| <code>BatchDetail()</code>                                              | Implements batch detail for the BatchDetail area. Source behavior and authorization are enforced by the surrounding module and database policies.          | src/Teacher/BatchDetail.jsx:109  |
| <code>BatchList()</code>                                                | Implements batch list for the BatchList area. Source behavior and authorization are enforced by the surrounding module and database policies.              | src/Teacher/BatchList.jsx:97     |

| Function                                                    | Responsibility                                                                                                                                                       | Source                                  |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| <code>MiniStat({ label, value, active, onClick })</code>    | Implements mini stat for the BatchList area. Source behavior and authorization are enforced by the surrounding module and database policies.                         | src/Teacher/BatchList.jsx:545           |
| <code>CourseCategorySelect({ value = "", ...props })</code> | Implements course category select for the CourseCategorySelect area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/Teacher/CourseCategorySelect.jsx:10 |
| <code>CourseContent()</code>                                | Implements course content for the CourseContent area. Source behavior and authorization are enforced by the surrounding module and database policies.                | src/Teacher/CourseContent.js x:71       |
| <code>LessonRow({})</code>                                  | LESSON ROW                                                                                                                                                           | src/Teacher/CourseContent.js x:1856     |
| <code>Modal({})</code>                                      | MODAL                                                                                                                                                                | src/Teacher/CourseContent.js x:1987     |
| <code>LessonTypeCard({})</code>                             | TYPE CARD                                                                                                                                                            | src/Teacher/CourseContent.js x:2027     |
| <code>StatusBadge({ status })</code>                        | STATUS                                                                                                                                                               | src/Teacher/CourseContent.js x:2082     |
| <code>IconButton({})</code>                                 | ICON BUTTON                                                                                                                                                          | src/Teacher/CourseContent.js x:2128     |
| <code>LoadingScreen()</code>                                | LOADING                                                                                                                                                              | src/Teacher/CourseContent.js x:2157     |
| <code>StatusBadge({ status })</code>                        | STATUS BADGE                                                                                                                                                         | src/Teacher/Courses.jsx:166             |
| <code>TypeBadge({ type })</code>                            | TYPE BADGE                                                                                                                                                           | src/Teacher/Courses.jsx:187             |
| <code>TeacherCourses()</code>                               | MAIN COMPONENT                                                                                                                                                       | src/Teacher/Courses.jsx:214             |
| <code>MiniStat({})</code>                                   | MINI STAT                                                                                                                                                            | src/Teacher/Courses.jsx:700             |
| <code>CourseRow({})</code>                                  | COURSE ROW                                                                                                                                                           | src/Teacher/Courses.jsx:736             |
| <code>LoadingState()</code>                                 | LOADING                                                                                                                                                              | src/Teacher/Courses.jsx:999             |
| <code>EmptyCourses({})</code>                               | EMPTY                                                                                                                                                                | src/Teacher/Courses.jsx:1033            |
| <code>CreateCourse()</code>                                 | Creates course for the CreateCourse area. Source behavior and authorization are enforced by the surrounding module and database policies.                            | src/Teacher/CreateCourse.jsx :56        |
| <code>EditCourse()</code>                                   | Implements edit course for the EditCourse area. Source behavior and authorization are enforced by the surrounding module and database policies.                      | src/Teacher/EditCourse.jsx:22           |
| <code>OfflineEnrollments()</code>                           | Implements offline enrollments for the OfflineEnrollments area. Source behavior and authorization are enforced by the surrounding module and database policies.      | src/Teacher/OfflineEnrollment s.jsx:7   |
| <code>AdminDashboard()</code>                               | Implements admin dashboard for the AdminDashboard area. Source behavior and authorization are enforced by the surrounding module and database policies.              | src/Admin/AdminDashboard.js x:331       |
| <code>Admins()</code>                                       | Implements admins for the Admins area. Source behavior and authorization are enforced by the surrounding module and database policies.                               | src/Admin/Admins.jsx:87                 |
| <code>Pill({ tone })</code>                                 | Implements pill for the AdminUI area. Source behavior and authorization are enforced by the surrounding module and database policies.                                | src/Admin/AdminUI.jsx:38                |
| <code>StatCard({})</code>                                   | Implements stat card for the AdminUI area. Source behavior and authorization are enforced by the surrounding module and database policies.                           | src/Admin/AdminUI.jsx:116               |

| Function                                                | Responsibility                                                                                                                                            | Source                           |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <code>Card({})</code>                                   | Implements card for the AdminUI area. Source behavior and authorization are enforced by the surrounding module and database policies.                     | src/Admin/AdminUI.jsx:227        |
| <code>Modal({})</code>                                  | Implements modal for the AdminUI area. Source behavior and authorization are enforced by the surrounding module and database policies.                    | src/Admin/AdminUI.jsx:309        |
| <code>Field({})</code>                                  | Implements field for the AdminUI area. Source behavior and authorization are enforced by the surrounding module and database policies.                    | src/Admin/AdminUI.jsx:427        |
| <code>PrimaryButton({})</code>                          | Implements primary button for the AdminUI area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/Admin/AdminUI.jsx:475        |
| <code>GhostButton({})</code>                            | Implements ghost button for the AdminUI area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/Admin/AdminUI.jsx:513        |
| <code>DangerButton({})</code>                           | Implements danger button for the AdminUI area. Source behavior and authorization are enforced by the surrounding module and database policies.            | src/Admin/AdminUI.jsx:551        |
| <code>EmptyState({})</code>                             | Implements empty state for the AdminUI area. Source behavior and authorization are enforced by the surrounding module and database policies.              | src/Admin/AdminUI.jsx:589        |
| <code>ConfirmDeleteModal({})</code>                     | Implements confirm delete modal for the AdminUI area. Source behavior and authorization are enforced by the surrounding module and database policies.     | src/Admin/AdminUI.jsx:616        |
| <code>Stat({ icon: Icon, label, value, detail })</code> | Implements stat for the Analytics area. Source behavior and authorization are enforced by the surrounding module and database policies.                   | src/Admin/Analytics.jsx:73       |
| <code>EmptyChart({ message })</code>                    | Implements empty chart for the Analytics area. Source behavior and authorization are enforced by the surrounding module and database policies.            | src/Admin/Analytics.jsx:90       |
| <code>Analytics()</code>                                | Implements analytics for the Analytics area. Source behavior and authorization are enforced by the surrounding module and database policies.              | src/Admin/Analytics.jsx:98       |
| <code>Announcements()</code>                            | Implements announcements for the Announcements area. Source behavior and authorization are enforced by the surrounding module and database policies.      | src/Admin/Announcements.jsx:293  |
| <code>StatCard({})</code>                               | Implements stat card for the Announcements area. Source behavior and authorization are enforced by the surrounding module and database policies.          | src/Admin/Announcements.jsx:1707 |
| <code>AnnouncementRow({})</code>                        | Implements announcement row for the Announcements area. Source behavior and authorization are enforced by the surrounding module and database policies.   | src/Admin/Announcements.jsx:1787 |
| <code>AnnouncementModal({})</code>                      | Implements announcement modal for the Announcements area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/Admin/Announcements.jsx:2234 |
| <code>AudienceOption({})</code>                         | Implements audience option for the Announcements area. Source behavior and authorization are enforced by the surrounding module and database policies.    | src/Admin/Announcements.jsx:3108 |
| <code>StatusOption({})</code>                           | Implements status option for the Announcements area. Source behavior and authorization are enforced by the surrounding module and database policies.      | src/Admin/Announcements.jsx:3185 |
| <code>LoadingState()</code>                             | Implements loading state for the Announcements area. Source behavior and authorization are enforced by the surrounding module and database policies.      | src/Admin/Announcements.jsx:3262 |

| Function                                    | Responsibility                                                                                                                                          | Source                           |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <code>EmptyState({})</code>                 | Implements empty state for the Announcements area. Source behavior and authorization are enforced by the surrounding module and database policies.      | src/Admin/Announcements.jsx:3307 |
| <code>StatusBadge({ status })</code>        | Implements status badge for the Courses area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/Admin/Courses.jsx:149        |
| <code>TypeBadge({ type })</code>            | Implements type badge for the Courses area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/Admin/Courses.jsx:165        |
| <code>Courses()</code>                      | MAIN COMPONENT                                                                                                                                          | src/Admin/Courses.jsx:195        |
| <code>CourseIdentity({ course })</code>     | COURSE IDENTITY                                                                                                                                         | src/Admin/Courses.jsx:1804       |
| <code>InstructorCell({ instructor })</code> | INSTRUCTOR CELL                                                                                                                                         | src/Admin/Courses.jsx:1871       |
| <code>CourseActions({})</code>              | COURSE ACTIONS                                                                                                                                          | src/Admin/Courses.jsx:1913       |
| <code>MobileCourseCard({})</code>           | MOBILE COURSE CARD                                                                                                                                      | src/Admin/Courses.jsx:2074       |
| <code>MobileInfo({ label, value })</code>   | MOBILE INFO                                                                                                                                             | src/Admin/Courses.jsx:2347       |
| <code>InfoItem({})</code>                   | INFO ITEM                                                                                                                                               | src/Admin/Courses.jsx:2370       |
| <code>ExportReports()</code>                | Implements export reports for the ExportReports area. Source behavior and authorization are enforced by the surrounding module and database policies.   | src/Admin/ExportReports.jsx:101  |
| <code>PaymentDetailsModal({})</code>        | Implements payment details modal for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/Admin/Payments.jsx:131       |
| <code>DetailRow({})</code>                  | Implements detail row for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies.            | src/Admin/Payments.jsx:303       |
| <code>StatCard({})</code>                   | Implements stat card for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/Admin/Payments.jsx:330       |
| <code>Payments()</code>                     | Implements payments for the Payments area. Source behavior and authorization are enforced by the surrounding module and database policies.              | src/Admin/Payments.jsx:370       |
| <code>Students()</code>                     | Implements students for the Students area. Source behavior and authorization are enforced by the surrounding module and database policies.              | src/Admin/Students.jsx:161       |
| <code>MiniStat({})</code>                   | Implements mini stat for the Students area. Source behavior and authorization are enforced by the surrounding module and database policies.             | src/Admin/Students.jsx:903       |
| <code>DesktopStudentRow({})</code>          | Implements desktop student row for the Students area. Source behavior and authorization are enforced by the surrounding module and database policies.   | src/Admin/Students.jsx:964       |
| <code>MobileStudentCard({})</code>          | Implements mobile student card for the Students area. Source behavior and authorization are enforced by the surrounding module and database policies.   | src/Admin/Students.jsx:1135      |
| <code>Avatar({ student })</code>            | Implements avatar for the Students area. Source behavior and authorization are enforced by the surrounding module and database policies.                | src/Admin/Students.jsx:1262      |
| <code>InfoBlock({})</code>                  | Implements info block for the Students area. Source behavior and authorization are enforced by the surrounding module and database policies.            | src/Admin/Students.jsx:1300      |

| Function                           | Responsibility                                                                                                                                         | Source                      |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| <code>ActionButton({}</code>       | Implements action button for the Students area. Source behavior and authorization are enforced by the surrounding module and database policies.        | src/Admin/Students.jsx:1331 |
| <code>MobileActionButton({}</code> | Implements mobile action button for the Students area. Source behavior and authorization are enforced by the surrounding module and database policies. | src/Admin/Students.jsx:1359 |
| <code>StudentEditModal({}</code>   | Implements student edit modal for the Students area. Source behavior and authorization are enforced by the surrounding module and database policies.   | src/Admin/Students.jsx:1390 |
| <code>StudentDetail({}</code>      | Implements student detail for the Students area. Source behavior and authorization are enforced by the surrounding module and database policies.       | src/Admin/Students.jsx:1514 |
| <code>DetailRow({}</code>          | Implements detail row for the Students area. Source behavior and authorization are enforced by the surrounding module and database policies.           | src/Admin/Students.jsx:1864 |

## Current SQL/RPC function definitions

| Function                                                                                                                                                                                                                  | Responsibility                                                                                                                                                                      | Source                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>lms_admin()</code>                                                                                                                                                                                                  | Implements admin for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                                     | supabase/migrations/202609200001_lms.sql:357              |
| <code>lms_admin_export_count(p_category text, p_course_id text default null, p_teacher_id uuid default null, p_payment_status text default null, p_student_status text default null, p_from timestampz default n)</code>  | Implements admin export count for the 202609240003_admin_export_count area. Source behavior and authorization are enforced by the surrounding module and database policies.         | supabase/migrations/202609240003_admin_export_count.sql:5 |
| <code>lms_admin_export_rows(p_category text, p_course_id text default null, p_teacher_id uuid default null, p_payment_status text default null, p_student_status text default null, p_from timestampz default n)</code>   | Implements admin export rows for the 202609240002_admin_exports area. Source behavior and authorization are enforced by the surrounding module and database policies.               | supabase/migrations/202609240002_admin_exports.sql:22     |
| <code>lms_admin_export_summary(p_from timestampz default null, p_to timestampz default null, p_course_id text default null, p_teacher_id uuid default null, p_payment_status text default null, p_student_status )</code> | Implements admin export summary for the 202609240002_admin_exports area. Source behavior and authorization are enforced by the surrounding module and database policies.            | supabase/migrations/202609240002_admin_exports.sql:114    |
| <code>lms_assignment_visible(aid text)</code>                                                                                                                                                                             | Implements assignment visible for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                        | supabase/migrations/202609200001_lms.sql:369              |
| <code>lms_complete_lesson(enrollment text, lesson text)</code>                                                                                                                                                            | Implements complete lesson for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                           | supabase/migrations/202609200001_lms.sql:742              |
| <code>lms_confirm_emi_installment(order_ref text, payment_ref text)</code>                                                                                                                                                | Implements confirm emi installment for the 20260925000000_lms_emi area. Source behavior and authorization are enforced by the surrounding module and database policies.             | supabase/migrations/20260925000000_lms_emi.sql:135        |
| <code>lms_confirm_payment(order_ref text, payment_ref text)</code>                                                                                                                                                        | Implements confirm payment for the 20260925000000_lms_emi area. Source behavior and authorization are enforced by the surrounding module and database policies.                     | supabase/migrations/20260925000000_lms_emi.sql:176        |
| <code>lms_course_community_click_count(cid text)</code>                                                                                                                                                                   | Implements course community click count for the 202609240001_course_community area. Source behavior and authorization are enforced by the surrounding module and database policies. | supabase/migrations/202609240001_course_community.sql:88  |
| <code>lms_course_member(cid text)</code>                                                                                                                                                                                  | Implements course member for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                             | supabase/migrations/202609200001_lms.sql:366              |
| <code>lms_course_outline(course text)</code>                                                                                                                                                                              | Implements course outline for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                            | supabase/migrations/202609200001_lms.sql:778              |
| <code>lms_discussion_member(did text)</code>                                                                                                                                                                              | Implements discussion member for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                         | supabase/migrations/202609200001_lms.sql:377              |
| <code>lms_enroll_free(course text)</code>                                                                                                                                                                                 | Enrolls free for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                                         | supabase/migrations/202609200001_lms.sql:727              |

| Function                                                                                                    | Responsibility                                                                                                                                                                                       | Source                                                                        |
|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>lms_enrolled(cid text)</code>                                                                         | Implements enrolled for the 202609230006_duration_based_emi area. Source behavior and authorization are enforced by the surrounding module and database policies.                                    | supabase/migrations/202609230006_duration_based_emi.sql:70                    |
| <code>lms_enrollment_created()</code>                                                                       | Implements enrollment created for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                                         | supabase/migrations/202609200001_lms.sql:784                                  |
| <code>lms_grant_offline_enrollment(p_course text, p_student uuid, p_amount bigint)</code>                   | Implements grant offline enrollment for the 202609230003_teacher_offline_enrollments area. Source behavior and authorization are enforced by the surrounding module and database policies.           | supabase/migrations/202609230003_teacher_offline_enrollments.sql:34           |
| <code>lms_grant_offline_enrollment_by_email(p_course text, p_email text, p_amount bigint)</code>            | Implements grant offline enrollment by email for the 202609230004_offline_email_admissions area. Source behavior and authorization are enforced by the surrounding module and database policies.     | supabase/migrations/202609230004_offline_email_admissions.sql:21              |
| <code>lms_guard_record()</code>                                                                             | Implements guard record for the 202609220001_teacher_publish area. Source behavior and authorization are enforced by the surrounding module and database policies.                                   | supabase/migrations/202609220001_teacher_publish.sql:2                        |
| <code>lms_is_course_teacher(cid text)</code>                                                                | Implements is course teacher for the 202609240001_course_community area. Source behavior and authorization are enforced by the surrounding module and database policies.                             | supabase/migrations/202609240001_course_community.sql:31                      |
| <code>lms_is_enrolled(cid text)</code>                                                                      | Implements is enrolled for the 202609240001_course_community area. Source behavior and authorization are enforced by the surrounding module and database policies.                                   | supabase/migrations/202609240001_course_community.sql:35                      |
| <code>lms_join_course_community(cid text)</code>                                                            | Implements join course community for the 202609240001_course_community area. Source behavior and authorization are enforced by the surrounding module and database policies.                         | supabase/migrations/202609240001_course_community.sql:74                      |
| <code>lms_manage_batch_student(action text, batch text, student uuid, destination text default null)</code> | Implements manage batch student for the 202609220004_teacher_batch_student_search area. Source behavior and authorization are enforced by the surrounding module and database policies.              | supabase/migrations/202609220004_teacher_batch_student_search.sql:44          |
| <code>lms_mark_course_engagement_attendance(course text)</code>                                             | Marks course engagement attendance for the 202609260005_automatic_course_engagement_attendance area. Source behavior and authorization are enforced by the surrounding module and database policies. | supabase/migrations/202609260005_automatic_course_engagement_attendance.sql:3 |
| <code>lms_media_access(object_name text, writing boolean)</code>                                            | Implements media access for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                                               | supabase/migrations/202609200001_lms.sql:836                                  |
| <code>lms_owns_course(cid text)</code>                                                                      | Implements owns course for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                | supabase/migrations/202609200001_lms.sql:360                                  |
| <code>lms_prepare_emi_plan(p_plan_id text)</code>                                                           | Implements prepare emi plan for the 20260925000000_lms_emi area. Source behavior and authorization are enforced by the surrounding module and database policies.                                     | supabase/migrations/20260925000000_lms_emi.sql:81                             |
| <code>lms_publish_notification()</code>                                                                     | Implements publish notification for the 202609230007_safe_publish_notifications area. Source behavior and authorization are enforced by the surrounding module and database policies.                | supabase/migrations/202609230007_safe_publish_notifications.sql:4             |
| <code>lms_reply_count()</code>                                                                              | Implements reply count for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                | supabase/migrations/202609200001_lms.sql:873                                  |
| <code>lms_role()</code>                                                                                     | Implements role for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                                                       | supabase/migrations/202609200001_lms.sql:354                                  |

| Function                                                                                         | Responsibility                                                                                                                                                                                 | Source                                                                  |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <code>lms_search_batch_students(p_course text, p_batch text, p_search text default '')</code>    | Implements search batch students for the 202609220004_teacher_batch_student_search area. Source behavior and authorization are enforced by the surrounding module and database policies.       | supabase/migrations/202609220004_teacher_batch_student_search.sql:4     |
| <code>lms_search_course_students(p_course text, p_search text default '')</code>                 | Implements search course students for the 202609230003_teacher_offline_enrollments area. Source behavior and authorization are enforced by the surrounding module and database policies.       | supabase/migrations/202609230003_teacher_offline_enrollments.sql:3      |
| <code>lms_shared_student(sid uuid)</code>                                                        | Implements shared student for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                                       | supabase/migrations/202609200001_lms.sql:374                            |
| <code>lms_sync_auth_profile()</code>                                                             | Synchronizes auth profile for the 202609250001_email_password_signup area. Source behavior and authorization are enforced by the surrounding module and database policies.                     | supabase/migrations/202609250001_email_password_signup.sql:3            |
| <code>lms_teacher_only_course_publish()</code>                                                   | Implements teacher only course publish for the 202609260003_teacher_only_course_publish area. Source behavior and authorization are enforced by the surrounding module and database policies.  | supabase/migrations/202609260003_teacher_only_course_publish.sql:3      |
| <code>lms_weekday_number(day_name text)</code>                                                   | Implements weekday number for the 20260925000000_lms_emi area. Source behavior and authorization are enforced by the surrounding module and database policies.                                 | supabase/migrations/20260925000000_lms_emi.sql:57                       |
| <code>lms_write_records(operations jsonb, reads jsonb default '[]')</code>                       | Implements write records for the 202609200001_lms area. Source behavior and authorization are enforced by the surrounding module and database policies.                                        | supabase/migrations/202609200001_lms.sql:672                            |
| <code>nth_session_date(start_on date, weekdays text[], occurrence integer)</code>                | Implements nth session date for the 20260925000000_lms_emi area. Source behavior and authorization are enforced by the surrounding module and database policies.                               | supabase/migrations/20260925000000_lms_emi.sql:64                       |
| <code>recompute_module_access(p_student uuid, p_course text, p_plan_id text default null)</code> | Implements recompute module access for the 202609260004_full_payment_unlocks_all_modules area. Source behavior and authorization are enforced by the surrounding module and database policies. | supabase/migrations/202609260004_full_payment_unlocks_all_modules.sql:3 |

## 10. Operational notes and test checklist

| Deployment order                 | Purpose / downstream behavior                                                                         |
|----------------------------------|-------------------------------------------------------------------------------------------------------|
| <b> -- Apply migrations</b>      | Run unapplied Supabase migrations in filename order before deploying client code that calls new RPCs. |
| <b> -- Deploy Edge Functions</b> | Publish payment, webhook, EMI, and cron functions with required secrets.                              |
| <b> -- Configure webhooks</b>    | Razorpay webhook secret and endpoint must match the deployed payment webhook.                         |
| <b> -- Deploy frontend</b>       | Build and publish the Vite application after backend compatibility is available.                      |
| <b> -- Smoke test roles</b>      | Test one student, teacher, and admin account with real RLS policies enabled.                          |

| High-value acceptance tests    | Result                                                                                |
|--------------------------------|---------------------------------------------------------------------------------------|
| <b>1. Full payment</b>         | Captured payment creates active/paid enrollment and every module is accessible.       |
| <b>2. EMI</b>                  | Registration and each installment unlock exactly the expected modules.                |
| <b>3. Automatic attendance</b> | Opening multiple lessons on one day produces exactly one present record.              |
| <b>4. Batch attendance</b>     | Teacher save produces student-visible present/late/absent records.                    |
| <b>5. Assignments</b>          | Batch targeting and grading permissions behave correctly.                             |
| <b>6. Course publishing</b>    | Teacher cannot directly publish; admin approval succeeds.                             |
| <b>7. Refund</b>               | Pending refund retains access; processed eligible refund revokes access.              |
| <b>8. Cross-role privacy</b>   | Unrelated students and teachers cannot read or mutate each other's protected records. |

Catalog totals: 119 primary UI components, 229 reusable/auth/server JavaScript functions, and 38 current SQL/RPC functions.

### End of document